

Diagrama de la arquitectura de la solución

Arquitectura de la Solución - Sistema CableNet

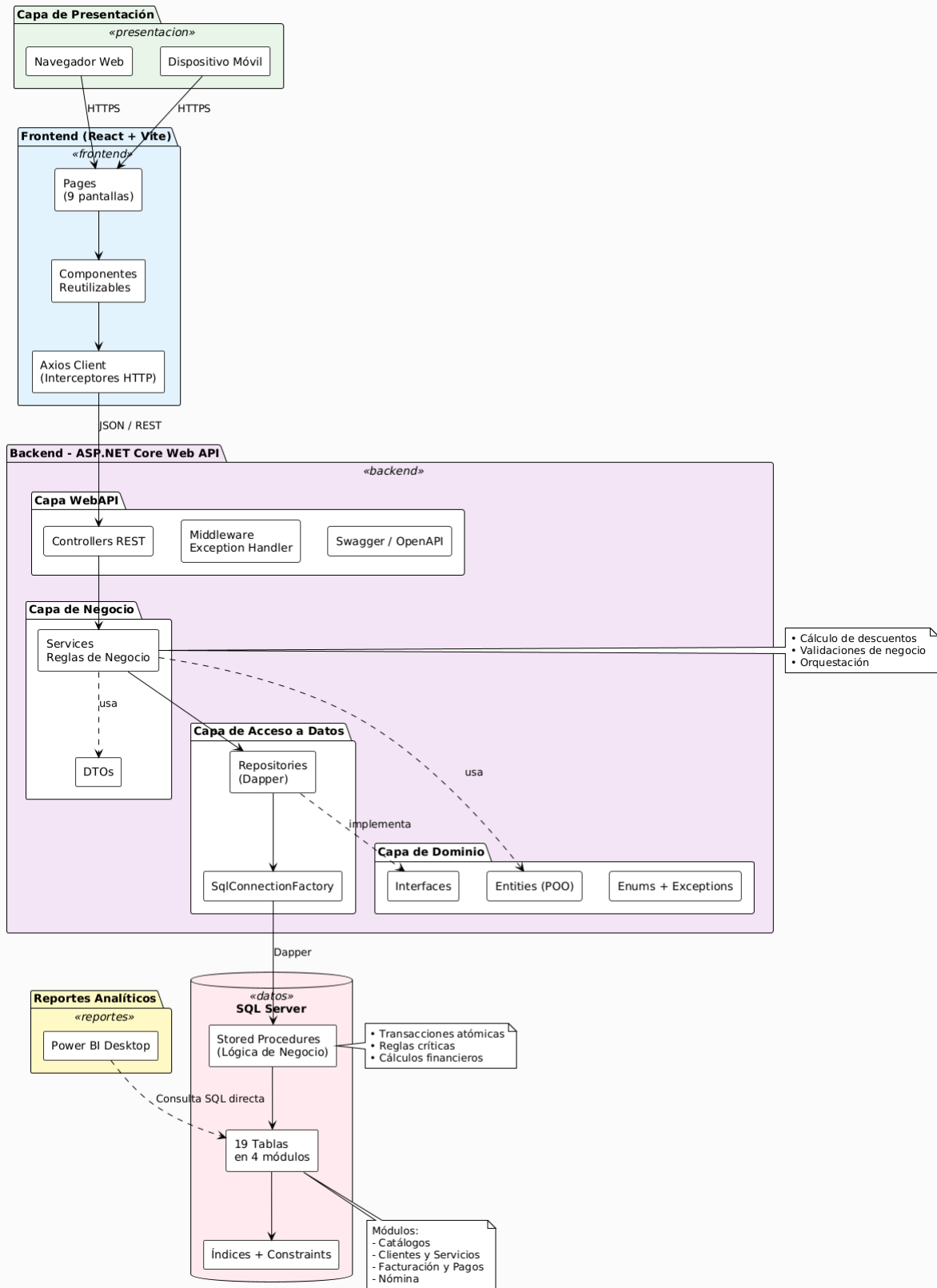
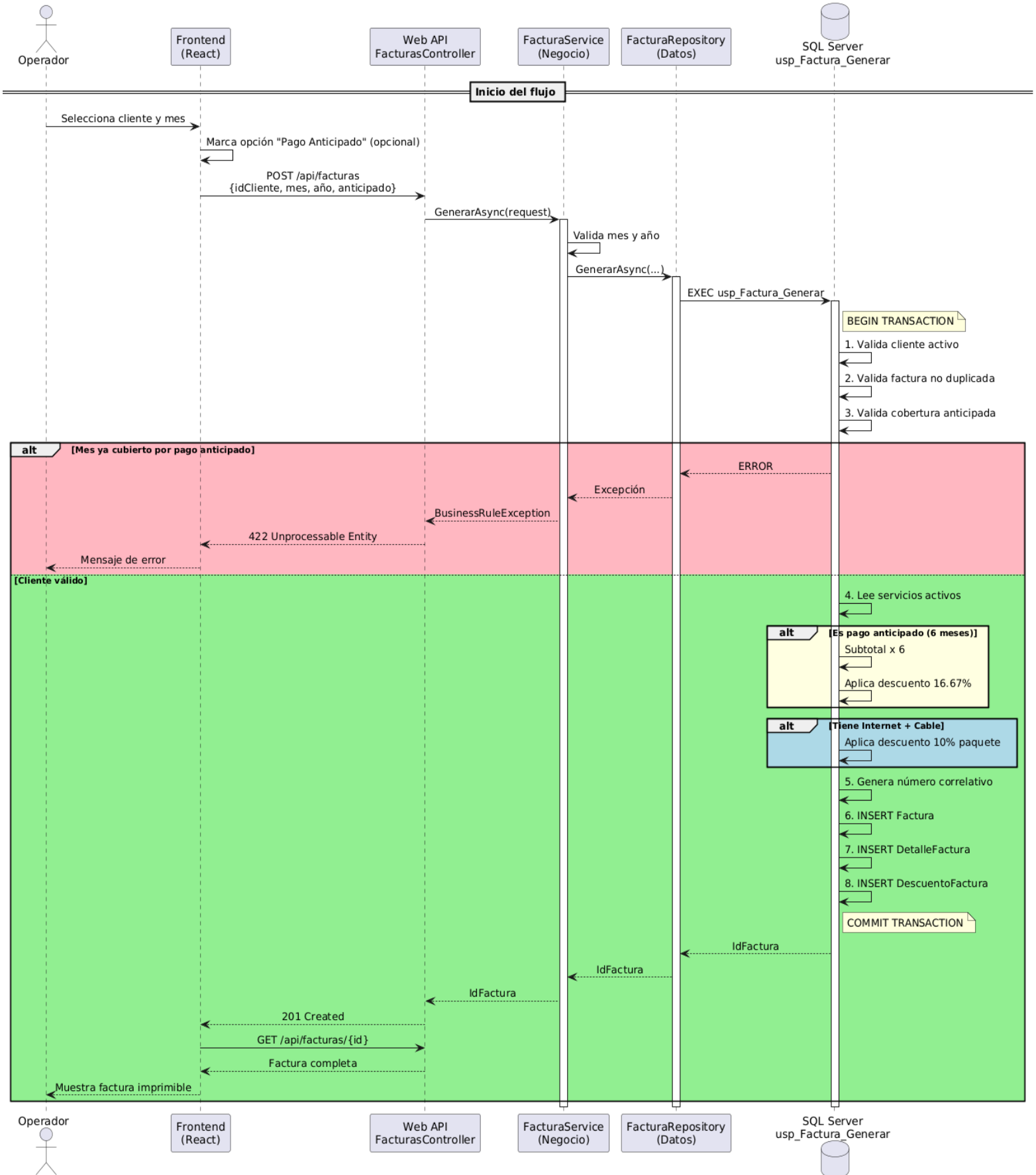
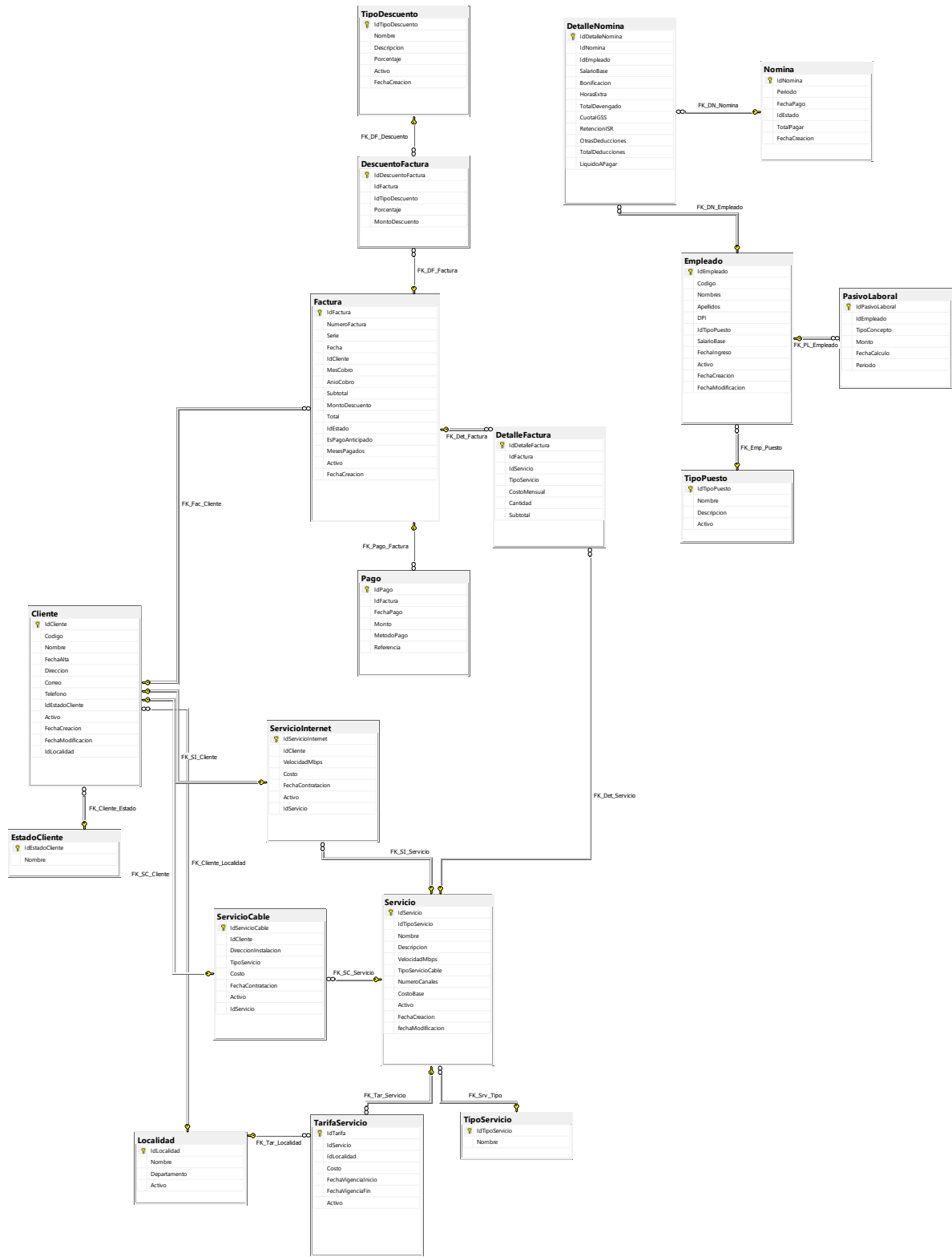


Diagrama de Secuencia UML

Flujo de Facturación Mensual con Reglas de Negocio





Liste los requerimientos funcionales y no funcionales

Requerimientos Funcionales

ID	MÓDULO	DESCRIPCIÓN DEL REQUERIMIENTO	PRIORIDAD (MOSCOW)
RF-01	Clientes	Registrar, modificar, consultar y dar de baja clientes (borrado lógico).	Must
RF-02	Clientes	Validar que el código de cliente sea único.	Must
RF-03	Clientes	Asignar estado al cliente: Activo, Suspendido, Moroso, Inactivo.	Must
RF-04	Clientes	Asociar un cliente a una localidad (para cálculo de tarifas).	Must
RF-05	Servicios	Mantener catálogo configurable de servicios (Internet 15/25/50 Mbps, Cable Básico/Premium).	Must
RF-06	Servicios	Permitir configurar costos por localidad y vigencia.	Must
RF-07	Servicios	Permitir configurar tipos de descuento (porcentaje, nombre, descripción).	Must
RF-08	Contratación	Permitir asignar Internet, Cable o ambos servicios a un cliente.	Must
RF-09	Contratación	Aplicar automáticamente 10% de descuento si el cliente tiene ambos servicios.	Must
RF-10	Contratación	Calcular costo total del contrato considerando descuentos aplicables.	Must
RF-11	Facturación	Generar factura mensual por cliente con número correlativo y serie.	Must
RF-12	Facturación	Almacenar detalle de servicios facturados (snapshot del costo pactado).	Must
RF-13	Facturación	Permitir generar factura de pago anticipado de 6 meses (cobra 5, regala 1).	Must
RF-14	Facturación	Bloquear generación de factura cuando el mes está cubierto por pago anticipado.	Must
RF-15	Facturación	Anular facturas que no tengan pagos registrados.	Must
RF-16	Pagos	Registrar pagos parciales o completos contra una factura.	Must

RF-17	Pagos	Cambiar factura a estado "Pagada" cuando los pagos cubren el total.	Must
RF-18	Pagos	Validar que el monto no exceda el saldo pendiente.	Must
RF-19	Pagos	Mantener historial de pagos por cliente.	Must
RF-20	Morosidad	Detectar clientes con 2+ meses sin pagar (alerta) o 3+ meses (suspensión).	Must
RF-21	Morosidad	Al detectar 3 meses sin pagar, bloquear facturas y suspender servicios automáticamente.	Must
RF-22	Morosidad	Reactivar cliente cuando pague todas sus deudas pendientes.	Must
RF-23	Upgrade Automático	Detectar clientes de Internet con 1+ años pagando facturas a tiempo.	Could
RF-24	Upgrade Automático	Aumentar velocidad +5 Mbps anuales hasta tope de 50 Mbps.	Could
RF-25	Upgrade Automático	Permitir aplicar upgrade individual o masivo.	Could
RF-26	Alarma y Reportes	Generar alarma mensual con clientes que deben pagar el mes.	Should
RF-27	Alarma y Reportes	Mostrar reportes: clientes activos, suspendidos, morosos, ganancias por mes.	Should
RF-28	Alarma y Reportes	Imprimir reportes y facturas en formato A4.	Should
RF-29	Nómina	Mantener registro de empleados con tipo de puesto y salario base.	Could
RF-30	Nómina	Calcular nómina mensual con IGSS (4.83%), ISR estimado y bonificación de ley.	Could
RF-31	Nómina	Provisionar pasivo laboral (aguinaldo, bono 14, vacaciones, indemnización).	Could

Requerimientos No Funcionales

CATEGORÍA	ESPECIFICACIÓN TÉCNICA	JUSTIFICACIÓN ARQUITECTÓNICA
SEGURIDAD	Autenticación basada en roles (RBAC), contraseñas con hash seguro y uso obligatorio de HTTPS. Prevención de SQL Injection mediante SPs paramétricos.	Garantiza la integridad de los datos financieros y la privacidad de los clientes.
RENDIMIENTO	Consultas en < 2 segundos, soporte de 100 usuarios concurrentes y facturación masiva en < 5 minutos.	Optimización de recursos en el servidor de base de datos.
DISPONIBILIDAD	Disponibilidad del 99.5%, RTO < 4 horas y políticas de respaldos diarios automáticos de la base de datos.	Continuidad del negocio ante fallos de infraestructura o energía.
MANTENIBILIDAD	Arquitectura limpia en capas con una cobertura mínima de pruebas unitarias en la lógica de negocio.	Facilita la escalabilidad del código y la integración de nuevos desarrolladores.

Plan de Trabajo Propuesto

Fase	Actividad	Duración	Recursos
Fase 1	Análisis y diseño	2 semanas	1 Arquitecto + 1 Analista funcional
Fase 2	Diseño de base de datos y modelo de dominio	1 semana	1 DBA + 1 Arquitecto
Fase 3	Desarrollo Backend (API + SPs)	4 semanas	2 Desarrolladores Sr
Fase 4	Desarrollo Frontend (React)	3 semanas	1 Desarrollador Frontend
Fase 5	Integración y pruebas	2 semanas	1 QA + equipo dev
Fase 6	UAT con cliente y ajustes	1 semana	Equipo completo + cliente
Fase 7	Despliegue y capacitación	1 semana	DevOps + Capacitador

Cronograma Gantt

[illegible]

DERCAS

Contexto y Alcance del Proyecto

- Información general: Sponsor, responsables y control de versiones.
- Problema a resolver: El objetivo medible del negocio.
- Límites del sistema: Definición clara de qué está incluido (In-Scope) y qué no (Out-of-Scope).

Especificación de Requerimientos

- Funcionales: Casos de uso, historias de usuario y criterios de aceptación claros.
- No Funcionales: Expectativas de rendimiento, seguridad, concurrencia y disponibilidad.

Lógica y Reglas de Negocio

- Restricciones: Validaciones obligatorias del sistema.
- Algoritmos críticos: Fórmulas exactas (ej. cálculo de recargos por morosidad, aplicación de descuentos de paquete doble).

Arquitectura y Diseño Técnico

- Diseño de Datos: Diagrama Entidad-Relación (ER) y diccionario de datos.
- Stack Tecnológico: Patrones a utilizar (ej. Arquitectura en capas) y herramientas.
- Interfaces (UI/UX): Mockups o wireframes de las pantallas principales.

Integraciones y Dependencias

- Sistemas externos: APIs de terceros, pasarelas de pago o servicios web necesarios para que el flujo funcione.

Criterios de Calidad (QA) y Aceptación

- Plan de pruebas: Estrategia de validación y casos de prueba críticos.
- Criterio de salida: Qué condiciones exactas se deben cumplir para dar el desarrollo por "Terminado" y pasarlo a producción.

Cómo DevOps Apoyaría el Proyecto

Considero que implementar prácticas DevOps es fundamental para este proyecto porque nos permite automatizar procesos repetitivos y reducir drásticamente los errores humanos al momento de liberar nuevas versiones. Específicamente, apoyaría en:

Automatización de Despliegues (CI/CD): En lugar de copiar archivos o actualizar servidores manualmente, configuraríamos un *pipeline* (por ejemplo, con GitHub Actions o Azure DevOps). Al terminar de programar una mejora, el código se compila y se publica automáticamente. Esto agiliza la entrega y evita el clásico "en mi máquina sí funciona".

Separación de Ambientes: Nos asegura tener entornos aislados (Desarrollo, QA y Producción) que sean idénticos en su configuración. Así garantizamos que lo que se prueba internamente se comportará igual al liberarlo a los clientes.

Monitoreo Proactivo: En lugar de enterarnos de los errores porque el cliente llama a quejarse, tendríamos alertas automáticas. Si el proceso masivo de facturación falla o un *Stored Procedure* se queda colgado, el equipo se entera de inmediato.

Capacidad de Reacción (Rollbacks): Si una actualización rompe algo en producción, el control de versiones nos permite dar marcha atrás y regresar a la versión estable anterior en minutos.

Propuesta de Aseguramiento de Calidad (QA)

El enfoque de calidad para este sistema no se basará únicamente en pruebas manuales al final del desarrollo, sino en una estrategia transversal de prevención de errores enfocada en los módulos críticos del negocio.

Estrategia de Pruebas por Capas

- **Pruebas Unitarias (Backend):** Se implementarán pruebas automatizadas utilizando *xUnit* o *NUnit* para la capa de negocio en .NET. El objetivo principal será validar la lógica matemática crítica (cálculos de IGSS, aplicación de descuentos del 10% y recargos).
- **Pruebas de Integración:** Validaremos que la comunicación entre la API (.NET), Dapper y los *Stored Procedures* en SQL Server funcione correctamente sin pérdida de datos.
- **Pruebas de Interfaz (Frontend):** Se realizarán pruebas de los flujos de usuario en React para garantizar que las pantallas de facturación y asignación de servicios respondan correctamente.

Enfoque Basado en Riesgos Dado que el sistema maneja dinero y servicios, los esfuerzos de QA tendrán una revisión estricta en tres módulos críticos:

- **Facturación:** Evitar que se generen facturas duplicadas o con montos erróneos.
- **Pagos:** Asegurar que los abonos se registren exactamente y actualicen el saldo.
- **Morosidad:** Validar que la suspensión automática de servicios (a los 3 meses) no afecte por error a clientes solventes.

Criterios de Aceptación y Salida a Producción Ningún módulo se considerará "Terminado" ni pasará a producción a menos que cumpla con lo siguiente:

- Cero defectos bloqueantes o críticos (ej. errores que impidan cobrar o facturar).
- Lógica de negocio validada al 100% según los requerimientos solicitados.
- Aprobación final del usuario (UAT) tras probar el sistema con datos de prueba realistas.

PRÁCTICA DE DESARROLLO

Realice los siguientes queries de acuerdo con el diagrama ER que se le proporciona.

- a. Top 10 de clientes que tienen mayor peso bruto generado por producto y por mes

SELECT TOP 10

c.cliente AS Nombre_Cliente,
c.direccion AS Direccion,
c.nit AS NIT,
p.producto AS Producto,
SUM(e.pbruto) AS Peso_Bruto_Total,
MONTH(e.fecha) AS Mes,
YEAR(e.fecha) AS Anio

FROM dbo.envio e

INNER JOIN dbo.cliente c ON c.codcliente = e.codcliente

INNER JOIN dbo.producto p ON p.codproducto = e.codproducto

WHERE YEAR(e.fecha) = 2014

GROUP BY

c.cliente,
c.direccion,
c.nit,
p.producto,
MONTH(e.fecha),
YEAR(e.fecha)

ORDER BY

SUM(e.pbruto) DESC;

- b. Generar un detalle por finca del número[conteo] de transacciones[envios] mensuales durante el primer semestre del 2015, excluir aquellos conteos inferiores a 100, ordenar por finca.

```
SELECT
    f.finca          AS Finca,
    e.empresa        AS Empresa,
    COUNT(en.idenvio) AS Total_Transacciones,
    MONTH(en.fecha)  AS Mes,
    YEAR(en.fecha)    AS Anio
FROM dbo.envio en
INNER JOIN dbo.finca f ON f.codfinca = en.codfinca
                    AND f.codempresa = en.codempresa
INNER JOIN dbo.empresa e ON e.codempresa = f.codempresa
WHERE YEAR(en.fecha) = 2015
    AND MONTH(en.fecha) BETWEEN 1 AND 6
GROUP BY
    f.finca,
    e.empresa,
    MONTH(en.fecha),
    YEAR(en.fecha)
HAVING
    COUNT(en.idenvio) >= 15
ORDER BY
    f.finca,
    MONTH(en.fecha);
```

